

C# Abstract Class vs Interface

Experienced .NET Interview Guide

A practical interview-preparation reference covering concepts, differences, enterprise examples, DI, modern C# interfaces, SOLID, and senior-level answers.

1. Interview-ready explanation

Abstract class: A base class for related classes that share common behavior, common state, or common implementation. It cannot be instantiated directly. It can contain abstract members, concrete methods, virtual methods, constructors, fields, properties, and protected members.

Interface: Primarily defines a contract or capability that a class agrees to provide. A class can implement multiple interfaces, making interfaces particularly useful for loose coupling, Dependency Injection, testing, and interchangeable implementations.

Decision rule: If I need to define *what an object can do*, I generally prefer an interface. If I need to define *what an object fundamentally is* and share state or implementation among closely related classes, I consider an abstract class.

2. Abstract class example

```
public abstract class PaymentProcessor
{
    protected readonly ILogger _logger;

    protected PaymentProcessor(ILogger logger)
    {
        _logger = logger;
    }

    public decimal TransactionFee { get; protected set; }

    public abstract Task<bool> ProcessPaymentAsync(decimal amount);

    public virtual void LogTransaction(decimal amount)
    {
        _logger.Log($"Payment processed: {amount}");
    }

    protected bool ValidateAmount(decimal amount)
    {
        return amount > 0;
    }
}
```

An abstract class is appropriate here because payment processors are closely related and can share logging, validation, transaction state, initialization, and mandatory extension points.

3. Interface example

```
public interface IPaymentProcessor
{
    Task<bool> ProcessPaymentAsync(decimal amount);
}

public class CreditCardPaymentProcessor : IPaymentProcessor
{
    public async Task<bool> ProcessPaymentAsync(decimal amount)
```

```

    {
        // Credit card processing
        return true;
    }
}

public class UpiPaymentProcessor : IPaymentProcessor
{
    public async Task<bool> ProcessPaymentAsync(decimal amount)
    {
        // UPI processing
        return true;
    }
}

```

The consumer can depend only on the contract:

```

public class OrderService
{
    private readonly IPaymentProcessor _paymentProcessor;

    public OrderService(IPaymentProcessor paymentProcessor)
    {
        _paymentProcessor = paymentProcessor;
    }

    public async Task PlaceOrder(decimal amount)
    {
        await _paymentProcessor.ProcessPaymentAsync(amount);
    }
}

// Program.cs
builder.Services.AddScoped<IPaymentProcessor,
    CreditCardPaymentProcessor>();

```

4. Abstract class vs interface

Feature	Abstract Class	Interface
Primary purpose	Base implementation + abstraction	Contract / capability
Direct instantiation	No	No
Constructors	Yes	No normal instance constructors
Instance fields/state	Yes	No normal instance fields
Common implementation	Excellent	Limited; default interface methods are possible
Access control	Public/protected/private etc.	Contract-oriented
Multiple inheritance	Only one base class	Multiple interfaces allowed
DI/service contract	Possible	Very common
Testing/mocking	Possible	Usually cleaner
Best fit	Closely related types with shared implementation/state	Capabilities and interchangeable implementations

5. Multiple inheritance

C# does not support inheriting from multiple classes, but a class can implement multiple interfaces.

```
// Invalid
public class MyService : ClassA, ClassB { }

// Valid
public class OrderService :
    BaseService,
    IOrderService,
    IValidator,
    IDisposable
{
}
```

Remember: One base class + multiple interfaces.

6. Real enterprise notification example

```
public interface INotificationService
{
    Task SendAsync(string message);
}

public class EmailNotificationService : INotificationService
{
    public Task SendAsync(string message)
    {
        // Email implementation
        return Task.CompletedTask;
    }
}

public class SmsNotificationService : INotificationService
{
    public Task SendAsync(string message)
    {
        // SMS implementation
        return Task.CompletedTask;
    }
}
```

An interface is natural because email, SMS, Teams, push notifications, and other providers may have very different implementations while exposing the same capability.

7. When an abstract class is better

Suppose every notification must validate the recipient, send the notification, write an audit entry, and log the result. The overall workflow is shared, while only the actual sending mechanism changes.

```
public abstract class NotificationServiceBase
{
    protected void ValidateRecipient(string recipient)
    {
        // Common validation
    }

    protected void WriteAuditLog(string message)
    {
        // Common audit logic
    }

    protected abstract Task SendCoreAsync(
        string recipient,
        string message);

    public async Task SendAsync(
        string recipient,
        string message)
    {
        ValidateRecipient(recipient);
    }
}
```

```

        await SendCoreAsync(recipient, message);
        WriteAuditLog(message);
    }
}

```

This is a typical use of the **Template Method Pattern**: the base class controls the common algorithm while subclasses implement specific steps.

8. Using interface and abstract class together

In enterprise applications, the two approaches often complement each other.

```

public interface INotificationService
{
    Task SendAsync(string recipient, string message);
}

public abstract class NotificationServiceBase
    : INotificationService
{
    protected readonly ILogger _logger;

    protected NotificationServiceBase(ILogger logger)
    {
        _logger = logger;
    }

    public async Task SendAsync(string recipient, string message)
    {
        Validate(recipient);
        await SendCoreAsync(recipient, message);
        Log(message);
    }

    protected abstract Task SendCoreAsync(
        string recipient,
        string message);

    protected virtual void Validate(string recipient)
    {
        if (string.IsNullOrEmpty(recipient))
            throw new ArgumentException("Recipient required");
    }

    private void Log(string message)
    {
        _logger.Log(message);
    }
}

```

Consumers depend on **INotificationService**, while concrete implementations can inherit shared behavior from **NotificationServiceBase**. This combines loose coupling with reusable implementation.

9. Which one is better?

Neither is universally better. They solve different design problems.

Prefer an interface when: you need a contract, implementations may be unrelated, you need multiple capabilities, you want loose coupling, you are defining DI boundaries, or you want easy substitution/mocking.

Typical examples: **IOrderService**, **IPaymentService**, **IEmailService**, **IRepository<T>**, **ICacheService**, **IStorageService**.

Prefer an abstract class when: derived classes are closely related, share state or protected helpers, require common constructor initialization, or need a partially fixed algorithm with overridable steps.

Typical examples: **DocumentProcessorBase**, **PaymentProcessorBase**, **ReportGeneratorBase**, **FileImporterBase**, **WorkflowHandlerBase**.

10. Repository scenario

Interview question: We have SQL Server, PostgreSQL, and MongoDB repositories. Would you use an interface or abstract class?

```
public interface IRepository<T>
{
    Task<T?> GetByIdAsync(Guid id);
    Task AddAsync(T entity);
    Task DeleteAsync(Guid id);
}
```

Start with an interface because the consumer cares about repository behavior rather than storage technology. If several EF Core repositories later share substantial implementation, introduce an abstract EF repository base class as well.

Senior-level principle: Interfaces and abstract classes are not competitors. They can complement each other.

11. Modern C#: interfaces can have implementation

```
public interface ILogger
{
    void Log(string message);

    void LogError(string message)
    {
        Log($"ERROR: {message}");
    }
}
```

Modern C# supports default interface implementations. Therefore, saying 'interfaces cannot contain implementation' is no longer completely correct.

However, default interface methods do not make interfaces a replacement for abstract base classes. Abstract classes remain appropriate for shared object state, constructors, protected implementation details, and richer inheritance hierarchies.

12. Can an abstract class have a constructor?

Yes. Its constructor initializes the base-class portion of a derived object.

```
public abstract class Employee
{
    protected Employee(string name)
    {
        Name = name;
    }

    public string Name { get; }
}

public class Developer : Employee
{
    public Developer(string name) : base(name)
    {
    }
}
```

13. Abstract, virtual, and normal methods

```
public abstract class ReportGenerator
{
    public void Validate()
```

```

    {
        Console.WriteLine("Common validation");
    }

    public abstract void Generate();

    public virtual void Export()
    {
        Console.WriteLine("Default export");
    }
}

```

Normal method: common implementation. **Abstract method:** derived class must implement it. **Virtual method:** has a default implementation and the derived class may override it.

14. Can an interface have a constructor?

An interface does not have a normal instance constructor because it does not represent an instantiated object with normal instance state. The implementing class owns construction and dependencies.

```

public interface IService
{
}

public class Service : IService
{
    private readonly AppDbContext _context;

    public Service(AppDbContext context)
    {
        _context = context;
    }
}

```

15. Interface Segregation Principle (ISP)

Avoid giant interfaces that force consumers to depend on operations they do not need. Prefer small, capability-oriented contracts.

```

public interface IUserService
{
    void CreateUser();
}

public interface IEmailSender
{
    void SendEmail();
}

public interface IReportGenerator
{
    void GenerateReport();
}

public class UserService :
    IUserService,
    IEmailSender
{
}

```

16. 60-second experienced interview answer

I use interfaces mainly to define contracts and achieve loose coupling. They work particularly well with Dependency Injection, unit testing, multiple implementations, and when otherwise unrelated classes need to expose the same capability.

I use an abstract class when I have a true base-class relationship and derived classes need shared state, constructor initialization, protected helper methods, or common implementation.

For example, **IPaymentProcessor** can define the contract for payment processing, while **PaymentProcessorBase** can contain common validation, logging, and audit functionality. Credit-card and UPI processors can inherit from the base class while consumers depend only on the interface.

So I do not consider one better than the other. I generally prefer interfaces at architectural boundaries and use abstract classes where there is genuine shared implementation and an inheritance relationship. I also prefer **composition over inheritance** unless inheritance clearly models the domain.

17. Quick decision rule

Need only a contract? -> Interface

Need common state or substantial common implementation among related classes? -> Abstract class

Need both? -> Interface as the public contract + abstract base class for shared implementation + concrete implementations.

For senior .NET interviews, be ready for the follow-up: **abstract class vs interface vs normal base class vs composition**. This usually tests design maturity more than C# syntax.